



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LEXICAL ANALYZER

Course: Compilers

Group: 05

Professor: M.C. René Adrian Dávila Pérez

Student(s):

Ambriz Cano Diego Emilio

Casillas Juárez Camila

Ferrer Cordero José Manuel

González Casimiro Juan Carlos

Vidrio López Mario Alexis

Contents

Introduction	2
Regular Expressions	2
Formal Definition of Each Token	3
Recognition Priority	5
NFA	6
NFA. Lexical Analyzer	6
NFA → DFA Conversion	7
Method, Using the Class Example	7
Conversion of the Complete NFA	8
Final DFA	10
Scanning Considerations	12
Lexer Process Diagrams	14
Scanning Loop	15
Lexical Analyzer Tests	16
T1. Valid Case: Declaration, Strings, and Operator	16
T2. Comments, Decimals, and Keywords	17
T3. Error Case: Malformed Character Constant	17
T4. Input for the Parser	18
CFG	18
Left factoring	19
FIRST, FOLLOW, LL(1) Table, and Trace.	19
Conclusions	21

Introduction

Every compilation process essentially consists of three elements. First, lexical, syntactic, and semantic analysis constitute the first point of contact with the source code; the second element is a symbol table, used to organize all the identifiers in the source code; the third is a synthesis phase, in which intermediate code is generated for the final execution of the program.

Under this premise, this work presents the development of the first element of a compiler, specifically the creation of a Lexical Analyzer. Its fundamental objective is to transform a continuous sequence of input characters into a series of syntactically meaningful units called tokens, while discarding non-significant elements such as whitespace and comments.

The result of the project is a Lexical Analyzer capable of producing a token output to validate a character string. In addition, this analyzer considers a list of 45 representative tokens for keywords, identifiers, constants of different types, string literals, operators, and punctuation delimiters.

Regular Expressions

The formal definition of the regular expression for each token class and for each discardable element is presented below.

letter $\rightarrow$ [a-zA-Z]
digit $\rightarrow$ [0-9]
escape $\rightarrow$ \ (any character except newline)
identifier $\rightarrow$ (letter | _) (letter | digit | _)*
keyword $\rightarrow$ an identifier lexeme that is in the lookup table
integer $\rightarrow$ digit⁺
decimal $\rightarrow$ digit⁺ . digit⁺
character $\rightarrow$ ' (any character except ' , \ and newline | escape) '
boolean $\rightarrow$ true | false (recognized as an identifier and looked up in the table)
constant $\rightarrow$ integer | decimal | character | boolean
literal $\rightarrow$ " (any character except " , \ and newline | escape)* "
operator $\rightarrow$ + | - | * | / | % | = | == | != | < | <= | > | >= | & | && | | | ||
punctuation $\rightarrow$ () | { | } | [|] | ; | ,
special $\rightarrow$ # | .
whitespace $\rightarrow$ (space | \t | \r | \n)⁺ discarded
comment $\rightarrow$ // (any character except newline)*
| / * ([^*] | *⁺[^* /])^{*} *⁺ / discarded

For identifiers, the first character may be a letter or an underscore, while the following characters may include letters, digits, or underscores. A decimal requires at least one digit after the point, so “5.” is not a complete constant.

The + and - signs are not considered part of numeric constants, but rather independent operators. In this way, an expression such as a-1 is recognized as an identifier, a subtraction operator, and an integer constant.

The “escape” token (the \ symbol) applies to strings and character constants; it allows the lexer to add quotation marks or any character other than newline to text strings without it being interpreted as the end of the string. For example, in "Hello \"John\"", a quotation mark preceded by the escape symbol does not mark the end of the string.

Tokens of type “Literal” and “Character” do not allow line breaks; consequently, the analyzer reports an error on the line where the string begins if it is not properly closed. Literals refer to strings enclosed in double quotes, "...", while characters are constants that contain exactly one element between single quotes ' '.

Boolean tokens (true and false) are words; they follow the identifier path and are distinguished in the lookup table, so they do not need states of their own. Block comments may contain any content until the closing sequence */ is found. The pattern must prevent an isolated asterisk from closing the comment incorrectly.

The analyzer reads the entire lexeme until it produces a valid token; even if it encounters characters or identifiers associated with a specific token, the lexeme is read completely. Finally, it is compared against the keywords and reserved operators.

These regular expressions serve as the basis for building the automata. In the implementation they will not be used directly through a regular expression library, since they will be represented by the DFA constructed later on.

Formal Definition of Each Token

The following is a list of 45 tokens grouped into keywords, identifiers, constants, literals, operators, punctuation, and special characters.

#	TokenType	Class (output)	Lexeme or pattern
1	INT	keyword	int
2	FLOAT	keyword	float
3	CHAR	keyword	char
4	DOUBLE	keyword	double
5	VOID	keyword	void
6	RETURN	keyword	return
7	IF	keyword	if
8	ELSE	keyword	else
9	WHILE	keyword	while

Table 1: Formal definition of each token.

#	TokenType	Class (output)	Lexeme or pattern
10	FOR	keyword	for
11	PRINT	keyword	print
12	ID	identifier	(letter _)(letter digit _)*
13	INT_CONST	constant	one or more digits
14	FLOAT_CONST	constant	digits, point, digits
15	CHAR_CONST	constant	one character or one escape between single quotes
16	TRUE	constant	true
17	FALSE	constant	false
18	STRING	literal	zero or more characters or escapes between double quotes
19	PLUS	operator	+
20	MINUS	operator	-
21	STAR	operator	*
22	SLASH	operator	/
23	PERCENT	operator	%
24	ASSIGN	operator	=
25	EQ	operator	==
26	NOT	operator	!
27	NEQ	operator	!=
28	LT	operator	<
29	LE	operator	<=
30	GT	operator	>
31	GE	operator	>=
32	AMP	operator	&
33	AND	operator	&&
34	BAR	operator	
35	OR	operator	
36	LPAREN	punctuation	(
37	RPAREN	punctuation	)
38	LBRACE	punctuation	{
39	RBRACE	punctuation	}
40	LBRACKET	punctuation	[
41	RBRACKET	punctuation	]
42	SEMI	punctuation	;
43	COMMA	punctuation	,
44	HASH	special	#
45	DOT	special	.

Table 1: Formal definition of each token.

Recognition Priority

When more than one pattern matches the input, token priority was determined by the following five rules.

1. **Maximal munch.** Take the longest lexeme that ends in a final state.
2. **Table before identifier.** If the lexeme ends in q_1 and is in the lookup table, the table wins. For example: `int` is a keyword and `true` is a constant.
3. **Non-tokens compete equally.** Whitespace and comments are recognized with the same rules and then discarded.
4. **Unclosed comment exception.** If the input ends inside a block comment (q_{10} or q_{11}), the scanner does not back up to the `/` of q_8 : an unclosed comment is reported.
5. **No final state, no token.** If no prefix reaches a final state, a lexical error is reported; there is no default token.

Maximal munch comes first because it makes the decision in most cases and **never conflicts with the other rules**. The table comes next because it only acts once the lexeme has already ended in q_1 . These rules resolve the ambiguity cases considered in the language defined for this assignment.

Rule four appears in the DFA. If `/* . . .` reaches the end without being closed, the last final state visited is q_8 ; without the exception, the scanner would back up and report a `/` operator instead of the actual error.

NFA

The following automaton was built together with our classmates during one of the course's lecture sessions. It serves as the theoretical basis for building the automaton of this project's lexical analyzer.

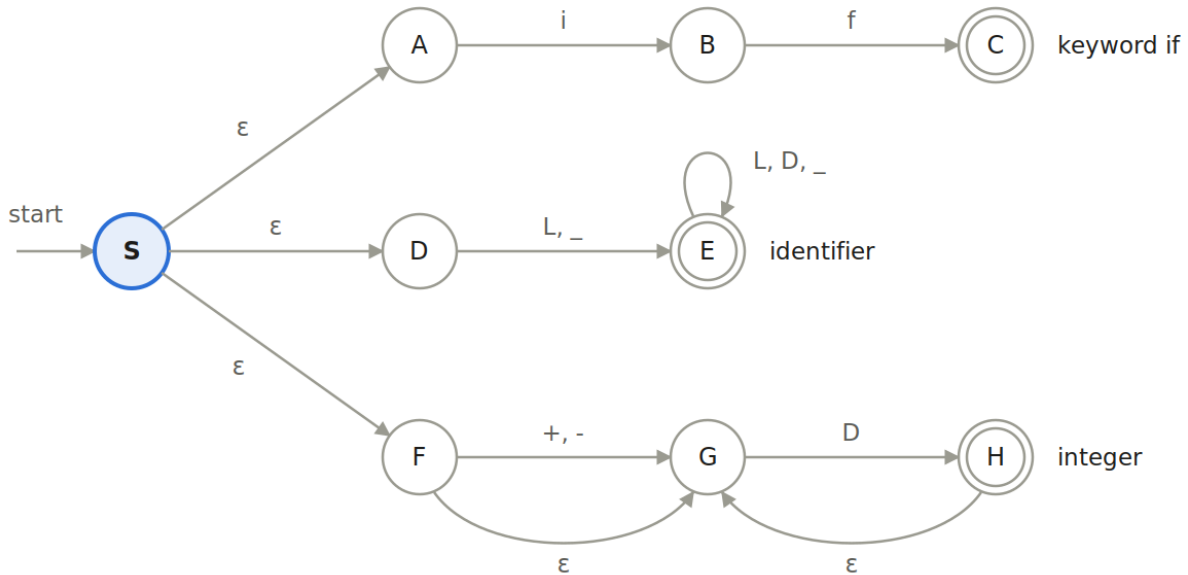


Figure 1: NFA built during the lecture sessions.

$$S \xrightarrow{\varepsilon} A \quad S \xrightarrow{\varepsilon} D \quad S \xrightarrow{\varepsilon} F$$

$$A \xrightarrow{i} B \quad B \xrightarrow{f} C^*$$

keyword if

$$D \xrightarrow{[a-zA-Z_]} E^* \quad E \xrightarrow{[a-zA-Z0-9_]} E$$

identifier

$$F \xrightarrow{+|-} G \quad F \xrightarrow{\varepsilon} G \quad G \xrightarrow{[0-9]} H^* \quad H \xrightarrow{\varepsilon} G$$

integer

(* = final state)

NFA. Lexical Analyzer

Each regular definition from the previous section was translated into a sub-automaton, that is, one state for each point in the pattern where a different type of character is expected, and a final state where the pattern can end. Then, S was joined to the nine initial states of the automaton from the previous section through ε -moves.

Sub-automaton	Transitions (source $\rightarrow$ symbol $\rightarrow$ target)	Final states
Start	$S \xrightarrow{\epsilon} I_0, N_0, T_0, K_0, C_0, O_0, P_0, X_0, W_0$	S : initial state
Identifier	$I_0 \xrightarrow{L} I_1^* \quad I_1 \xrightarrow{L D} I_1^*$	I_1 : identifier or keyword
Number	$N_0 \xrightarrow{D} N_1^* \quad N_1 \xrightarrow{D} N_1^* \quad N_1 \rightarrow N_2$ $N_2 \xrightarrow{D} N_3^* \quad N_3 \xrightarrow{D} N_3^*$	N_1 : integer N_3 : decimal
String	$T_0 \xrightarrow{''} T_1 \quad T_1 \xrightarrow{c} T_1 \quad T_1 \xrightarrow{\backslash} T_2$ $T_2 \xrightarrow{a} T_1 \quad T_1 \xrightarrow{''} T_3^*$	T_3 : literal (string)
Character	$K_0 \xrightarrow{' } K_1 \quad K_1 \xrightarrow{k} K_2 \quad K_1 \xrightarrow{\backslash} K_3$ $K_3 \xrightarrow{a} K_2 \quad K_2 \xrightarrow{' } K_4^*$	K_4 : character constant
Comments	$C_0 \xrightarrow{/} C_1 \quad C_1 \xrightarrow{/} C_2^* \quad C_2 \xrightarrow{a} C_2^*$ $C_1 \xrightarrow{*} C_3 \quad C_3 \xrightarrow{m} C_3 \quad C_3 \xrightarrow{*} C_4$ $C_4 \xrightarrow{*} C_4 \quad C_4 \xrightarrow{n} C_3 \quad C_4 \xrightarrow{/} C_5^*$	C_2 : line comment C_5 : block comment (both are discarded)
Operators	$O_0 \xrightarrow{+ - * \% /} O_1^* \quad O_0 \xrightarrow{= ! < >} O_2^* \quad O_2 \xrightarrow{=} O_3^*$ $O_0 \xrightarrow{\&} O_4^* \quad O_4 \xrightarrow{\&} O_3^* \quad O_0 \xrightarrow{ } O_5^*$ $O_5 \xrightarrow{ } O_3^*$	O_1, O_2, O_4, O_5 : one character O_3 : two characters
Punctuation	$P_0 \xrightarrow{\text{PUNCT}} P_1^*$	P_1 : punctuation
Special	$X_0 \xrightarrow{\#} X_1^*$	X_1 : special character
Whitespace	$W_0 \xrightarrow{\text{WS, NL}} W_1^* \quad W_1 \xrightarrow{\text{WS, NL}} W_1^*$	W_1 : whitespace (discarded)

Table 2. Transitions and final states for each sub-automaton of the Lexer.

Symbols

L = letter or $_$

D = digit

$\text{PUNCT} = () \{ \} [] ; ,$ WS = space

c = any character except $'$, $\backslash$ and NL

k = any character except $'$, $\backslash$ and NL

a = any character except NL NL = newline

m = any character except $*$

n = any character except $*$ and $/$

$[]$ = any of the listed symbols

The only nondeterminism: S , which opens all nine sub-automata at once via ϵ , and the character $/$, which leads both to O_1 (operator) and to C_1 (comment). Each sub-automaton, taken separately, is already deterministic.

NFA $\rightarrow$ DFA Conversion

A DFA has exactly one move per state and symbol, so it runs in a single pass with one table lookup per character. The resulting DFA can be implemented with a transition table, which determines the next state from the current state and the class of the character read.

Method, Using the Class Example

Two operations are used. ϵ -closure(T) is the set of states reachable from T using only ϵ ; move(T , a) is the set of states reachable from T on symbol a . Each DFA state is a set of NFA states.

1. $Q_0 = \varepsilon\text{-closure}(S)$. From S , ε leads to A , D and F ; from F , ε leads to G . $Q_0 = \{S, A, D, F, G\}$.
2. From Q_0 on i : A goes to B and D goes to E . Result $\{B, E\} = Q_1$.
3. From Q_0 on f , another letter, or $_$: only D moves, to E . Result $\{E\} = Q_2$.
4. From Q_0 on a digit: G goes to H , and via ε H returns to G . Result $\{G, H\} = Q_3$.
5. From Q_0 on $+$ or $-$: F goes to G . Result $\{G\} = Q_4$.
6. From Q_1 on f : B goes to C and E stays in E . Result $\{C, E\} = Q_5$.
7. From Q_1 on another letter or a digit: only E moves. Result $\{E\} = Q_2$.
8. Q_2 loops on itself; Q_3 and Q_4 go to Q_3 on a digit; Q_5 goes to Q_2 on a letter or digit (if x is an identifier).
9. A DFA state is final if it contains any NFA final state.

DFA state	NFA states	i	f	other letter, _	digit	+ or -	Final?
Q_0	$\{S, A, D, F, G\}$	Q_1	Q_2	Q_2	Q_3	Q_4	no
Q_1	$\{B, E\}$	Q_2	Q_5	Q_2	Q_2	–	identifier
Q_2	$\{E\}$	Q_2	Q_2	Q_2	Q_2	–	identifier
Q_3	$\{G, H\}$	–	–	–	Q_3	–	integer
Q_4	$\{G\}$	–	–	–	Q_3	–	no
Q_5	$\{C, E\}$	Q_2	Q_2	Q_2	Q_2	–	if (priority over identifier)

Table 3. State transitions for the conversion example.

Q_5 contains final states of two classes, C (if) and E (identifier); this is resolved with rule 2, “Table before identifier”. The result is the same as recognizing everything as an identifier and then querying a table.

Conversion of the Complete NFA

$\varepsilon\text{-closure}(S)$ gives q_0 , and each input symbol leads to a set of NFA states that becomes a DFA state:

DFA state	NFA set	DFA state	NFA set
q_0	$\{S, I_0, N_0, T_0, K_0, C_0, O_0, P_0, X_0, W_0\}$	q_{13}	$\{O_3\}$
q_1	$\{I_1\}$	q_{14}	$\{O_4\}$
q_2	$\{N_1\}$	q_{15}	$\{O_5\}$
q_3	$\{N_2\}$	q_{16}	$\{P_1\}$
q_4	$\{N_3\}$	q_{17}	$\{X_1\}$
q_5	$\{T_1\}$	q_{18}	$\{C_5\}$
q_6	$\{T_3\}$	q_{19}	$\{W_1\}$
q_7	$\{O_1\}$	q_{20}	$\{T_2\}$
q_8	$\{O_1, C_1\}$	q_{21}	$\{K_1\}$
q_9	$\{C_2\}$	q_{22}	$\{K_3\}$
q_{10}	$\{C_3\}$	q_{23}	$\{K_2\}$
q_{11}	$\{C_4\}$	q_{24}	$\{K_4\}$
q_{12}	$\{O_2\}$		

Table 4. State correspondence between the complete NFA and the resulting DFA.

Only q_0 and q_8 group more than one NFA state, because that is where the only nondeterminism was. The rest match one to one, which is why the DFA can be drawn as one row per class.

Final DFA

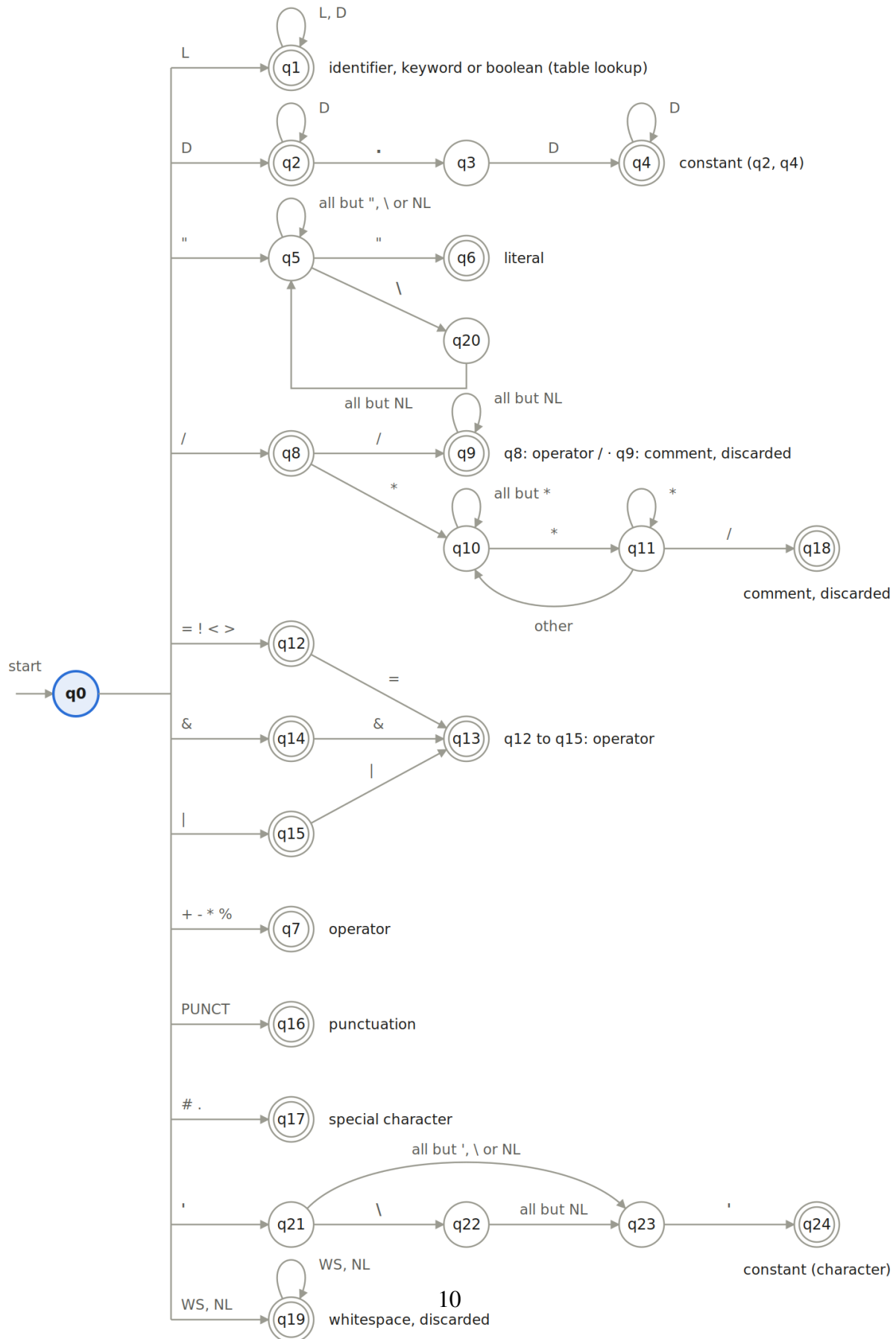


Figure 2: Lexer DFA · 25 states, 16 final

Each token class has its own row, so the final state where scanning stops determines the class of the lexeme.

Some operator states behave identically with respect to the input classes, so they can be represented by a single state. For example, the operators =, !, < and > share an intermediate state, since they can end on their own or continue with = to form a two-character operator. That is why = ! < > share q_{12} and all two-character operators end in q_{13} . q_{14} and q_{15} are not merged, because q_{14} advances on & and q_{15} on |.

Two characters go in the same column if they behave the same in every state. Thus, the 128 ASCII columns are reduced to 18. This grouping avoids defining a separate column for each ASCII character. For example, all letters behave the same way in the transitions associated with identifiers, so they are grouped into class L.

Class	Characters	Class	Characters
L	a-z, A-Z, _	AMP	&
D	0-9	BAR	
DOT	.	ARITH	+ - %
QUOTE	"	PUNCT	() { } [] ; ,
APOS	'	HASH	#
BSLASH	\	WS	space, tab, carriage return
SLASH	/	NL	newline
STAR	*	OTHER	any other character
EQ	=		
REL	! < >		

Table 5. Grouping of ASCII characters into equivalence classes for the DFA.

Transition table $\delta(\text{state}, \text{class})$; a dash means there is no move:

STATE	L	D	DOT	QUOTE	APOS	BSLASH	SLASH	STAR	EQ	REL	AMP	BAR	ARITH	PUNCT	HASH	WS	NL	OTHER
q_0	q_1	q_2	q_{17}	q_5	q_{21}	—	q_8	q_7	q_{12}	q_{12}	q_{14}	q_{15}	q_7	q_{16}	q_{17}	q_{19}	q_{19}	—
q_1	q_1	q_1	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_2	—	q_2	q_3	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_3	—	q_4	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_4	—	q_4	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_5	q_5	q_5	q_5	q_6	q_5	q_{20}	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	—	q_5
q_6	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_7	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_8	—	—	—	—	—	—	q_9	q_{10}	—	—	—	—	—	—	—	—	—	—
q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	q_9	—	q_9
q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{11}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}
q_{11}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{18}	q_{11}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}	q_{10}
q_{12}	—	—	—	—	—	—	—	—	q_{13}	—	—	—	—	—	—	—	—	—
q_{13}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{14}	—	—	—	—	—	—	—	—	—	—	q_{13}	—	—	—	—	—	—	—
q_{15}	—	—	—	—	—	—	—	—	—	—	—	q_{13}	—	—	—	—	—	—
q_{16}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{17}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{18}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{19}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	q_{19}	q_{19}	—
q_{20}	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	q_5	—	q_5
q_{21}	q_{23}	q_{23}	q_{23}	q_{23}	—	q_{22}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	—	q_{23}
q_{22}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	q_{23}	—	q_{23}
q_{23}	—	—	—	—	q_{24}	—	—	—	—	—	—	—	—	—	—	—	—	—
q_{24}	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—

Table 6. DFA transitions (25 states $\times$ 18 classes).

Final states (q_0 , q_3 , q_5 , q_{10} , q_{11} and q_{20} to q_{23} are not final):

Final state	Lexemes ending here	Returns
q_1	words	the TokenType from the lookup table; if not found, ID
q_2	10	INT_CONST
q_4	3.14	FLOAT_CONST
q_{24}	'a', '\n'	CHAR_CONST
q_6	"..."	STRING
$q_7, q_8, q_{12}, q_{13}, q_{14}, q_{15}$	+ - * % / = ! < > == != <= >= & &&	the operator according to the lexeme (PLUS, EQ, AND...)
q_{16}	() { } [] ; ,	the punctuation according to the lexeme (LPAREN, SEMI...)
q_{17}	# .	HASH or DOT
q_9, q_{18}	// ..., /* ... */	comment, discarded
q_{19}	spaces, tabs, newlines	whitespace, discarded

Table 7. Mapping of the DFA final states to their corresponding token types and lexemes.

Scanning Considerations

A lexical error occurs when scanning stops without having passed through a final state. For the constructed DFA, the main cases in which scanning can stop without producing a valid token occur in the following states:

Stuck state	Meaning	Message	Recovery
q_0	a character that does not start any token, such as @ or \	Lexical error 1:5: unexpected character '@'	skip that character
q_5 or q_{20}	string not closed on the same line	Lexical error 2:8: unterminated string literal	skip to the end of the line
q_{21} , q_{22} or q_{23}	malformed character constant: '', 'ab', or unclosed	Lexical error 3:10: malformed character constant	skip to the next ' on the line, or to the end of the line
q_{10} or q_{11} , at the end of the input	unclosed block comment (rule 4)	Lexical error 4:1: unterminated comment	stop scanning

Table 8. Lexical error handling.

There is no error in q_3 : 5. is always preceded by the final state q_2 , so 5 is emitted and scanning restarts from .. Recovery for character constants skips to the next quote so that 'ab' does not turn into a spurious identifier plus another error.

Lexer Process Diagrams

The flowchart shows how errors and discarded elements return to the start of the loop without adding tokens. The Lexer runs in 5 stages, shown in the following figure.

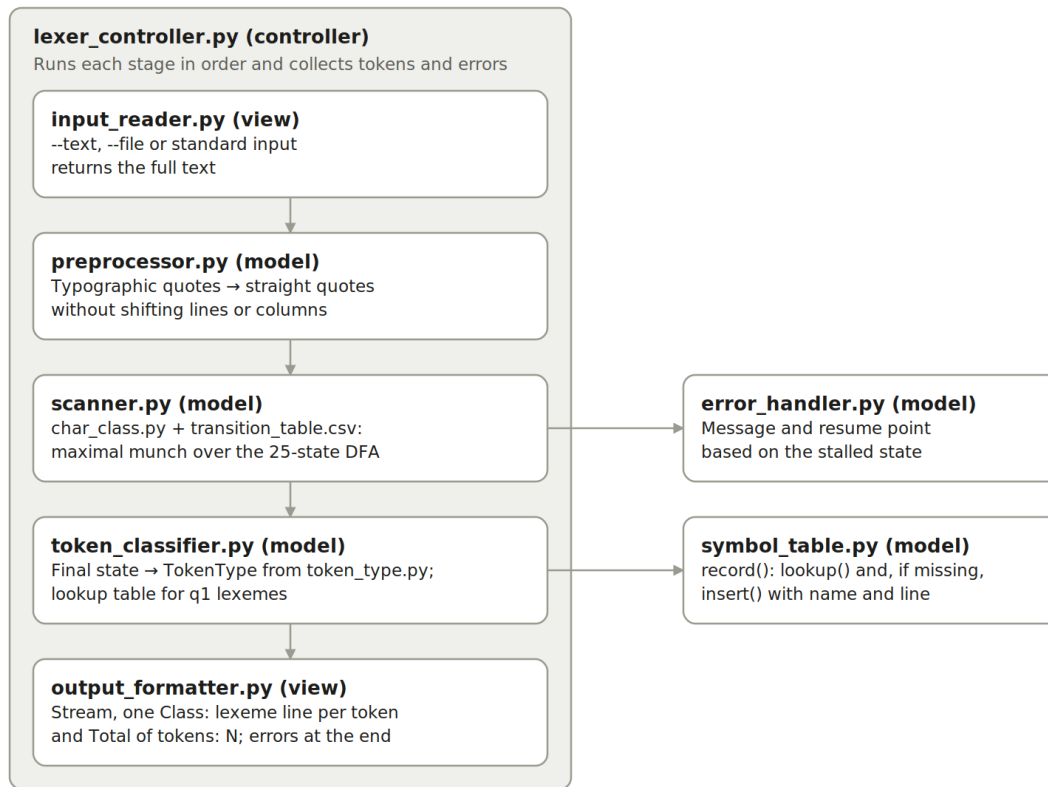


Figure 3: Lexer stages

Scanning Loop

The following figure shows a flowchart of the Lexer's scan process.

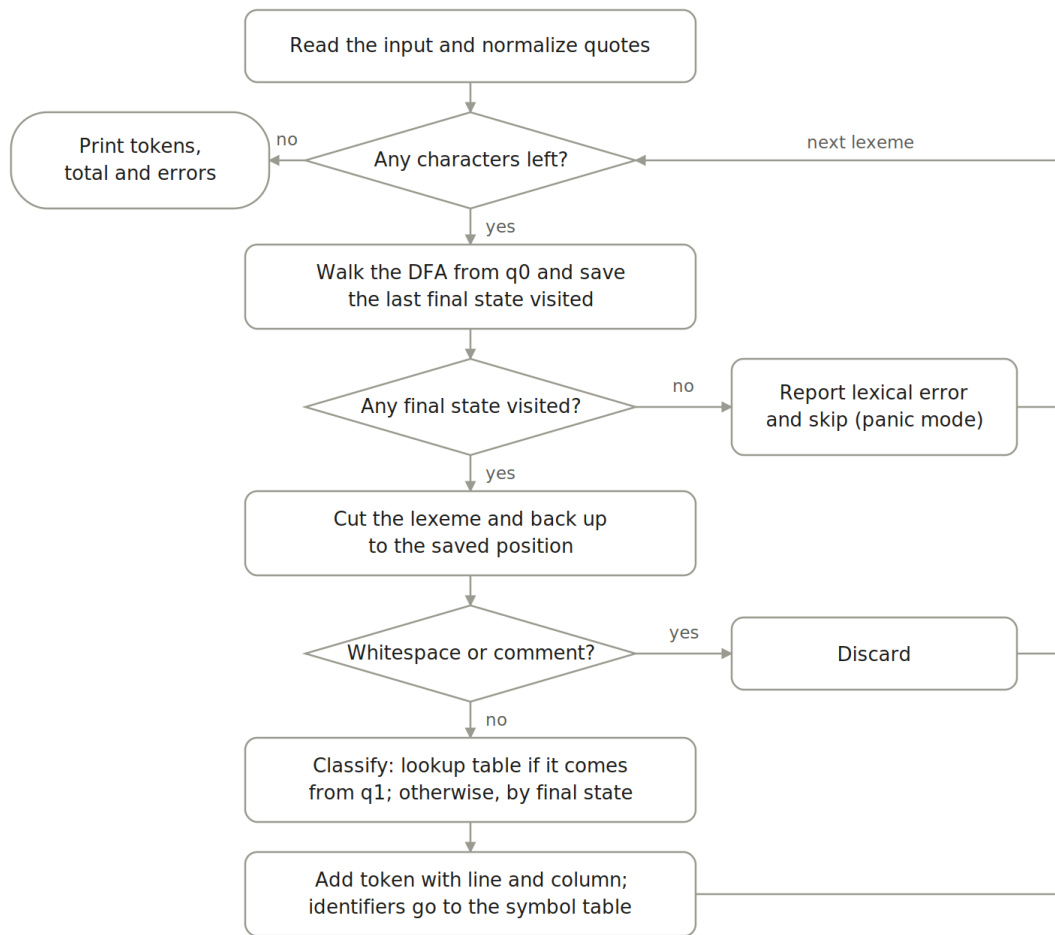


Figure 4: Scanning flowchart

Each iteration of the loop consumes one lexeme; errors and discarded elements return to the start without adding tokens.

This diagram was derived from the pseudocode: each if becomes a decision diamond, and each group of instructions becomes a box.

```
while pos < len(text):
    state = q0; i = pos; lastFinal = none; lastEnd = pos
    while i < len(text):
        next = delta[state][classOf(text[i])]
        if next == none: break
        state = next; i = i + 1
        if state in FINAL: lastFinal = state; lastEnd = i
    if i == len(text) and state in {q10, q11}:
        errors.report(line, col, "unterminated comment"); break
```

```

if lastFinal == none:
    errors.report(line, col, state); pos = recover(pos, state); continue
lexeme = text[pos : lastEnd]
kind = classify(lastFinal, lexeme)    # the lookup table is queried here
if kind not in {WHITESPACE, COMMENT}:
    tokens.add(kind, lexeme, line, col)
advanceLineCol(lexeme); pos = lastEnd

```

Lexical Analyzer Tests

The tests were run with the analyzer from the command line using `python main.py -text "..."`. The output has three parts: a line with the class of each token in order, one `Class: lexeme line` per token, and the total number of tokens (`Total of tokens: N`). Lexical errors are printed at the end in the format `Lexical error line:column: message` and are not counted as tokens. Comments and whitespace are discarded, so they do not appear in the output. Inner double quotes are written as `\"` because the string is enclosed in quotes on the console.

T1. Valid Case: Declaration, Strings, and Operator

Input with a declaration, two calls with strings, and the `&` operator:

```

unam.fi.compilers.g5.06>python main.py --text "int number; printf(\"Enter a
    number:\"); scanf(\"%d\", &number); "
keyword identifier punctuation identifier punctuation literal punctuation
    punctuation identifier punctuation literal punctuation operator identifier
    punctuation punctuation
Keyword: int
Identifier: number
Punctuation: ;
Identifier: printf
Punctuation: (
Literal: "Enter a number:"
Punctuation: )
Punctuation: ;
Identifier: scanf
Punctuation: (
Literal: "%d"
Punctuation: ,
Operator: &
Identifier: number
Punctuation: )
Punctuation: ;
Total of tokens: 16

```

Note that `int` is output as a keyword and `number` as an identifier (rule 2). `printf` shows maximal

munch at work (rule 1): although print is a keyword, the longest lexeme ending in q_1 is printf, which is not in the lookup table and is classified as ID. The strings "Enter a number:" and "%d" end in q_6 and are emitted in full, including their quotes. The isolated & ends in q_{14} and is a single-character operator, because it is not followed by another &.

T2. Comments, Decimals, and Keywords

Input with a decimal, a block comment, a string, and a line comment:

```
unam.fi.compilers.g5.06>python main.py --text "float x = 3.14; /* bloque */
    print(\"Hola\"); // fin"
keyword identifier operator constant punctuation keyword punctuation literal
    punctuation punctuation
Keyword: float
Identifier: x
Operator: =
Constant: 3.14
Punctuation: ;
Keyword: print
Punctuation: (
Literal: "Hola"
Punctuation: )
Punctuation: ;
Total of tokens: 10
```

The block comment (ending in q_{18}), the line comment (ending in q_9), and the whitespace (q_{19}) are recognized with the same rules as tokens but are discarded, so they do not appear in the output. The decimal 3.14 ends in q_4 and is emitted as a constant.

T3. Error Case: Malformed Character Constant

Input with a single quote that is never closed:

```
unam.fi.compilers.g5.06>python main.py --text "int number = 'a; printf(\"Enter a
    number:\"); scanf(\"%d\", &number); "
keyword identifier operator
Keyword: int
Identifier: number
Operator: =
Total of tokens: 3
Lexical error 1:14: malformed character constant
```

The DFA reads 'a and reaches q_{23} , but the next character is ;, and from q_{23} there is only a transition on '. Scanning stops without having passed through a final state, so the error is reported at line 1, column 14 (the position of the quote), with the message from Table 8 for q_{21} , q_{22} and q_{23} . Recovery

skips to the next ' on the line or, if there is none, to the end of the line. Here there is no other single quote, so the rest of the line is discarded, which is why only the three tokens before the error are emitted.

T4. Input for the Parser

Input with the declaration used in the CFG section:

```
unam.fi.compilers.g5.06>python main.py --text "int a = 10;"
keyword identifier operator constant punctuation
Keyword: int
Identifier: a
Operator: =
Constant: 10
Punctuation: ;
Total of tokens: 5
```

The five tokens correspond to INT, ID, ASSIGN, INT_CONST and SEMI, which are the terminals int, id, =, num and ; of grammar G_1 .

CFG

The following is the Context-Free Grammar $G_0 = (N, T, P, S)$ for the declaration, with $N = \{\text{Decl}, \text{Type}, \text{Value}\}$, $T = \{\text{int}, \text{float}, \text{char}, \text{id}, \text{num}, =, ;\}$ and $S = \text{Decl}$.

1. $\text{Decl} \rightarrow \text{Type id} ;$
2. $\text{Decl} \rightarrow \text{Type id} = \text{Value} ;$
3. $\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{char}$
4. $\text{Value} \rightarrow \text{num} \mid \text{id}$

A variable declaration was chosen because it shows a case of productions with a common prefix, which allows left factoring to be applied.

The token sequence `int id = num ;` was taken, and one production was written for each form of the declaration: without a value and with a value. The terminals are tokens from the following catalog:

Terminal	TokenType
int, float, char	INT, FLOAT, CHAR
id	ID
num	INT_CONST
=	ASSIGN
;	SEMI

There is one consideration for building the LL(1) parser. Productions 1 and 2 share the prefix Type id. After reading int a, a parser that does not account for this would not know which production to apply; therefore, in an LL(1) table both would fall into the cell [Decl, int].

Left factoring

The common prefix of G_0 was factored out, and the result, G_1 , was verified to be LL(1). To do so, the longest common prefix of the two Decl productions was identified: $\alpha = \text{Type id}$. What follows in each one is $\beta_1 = ;$ and $\beta_2 = \text{Value } ;$. The rule seen in class was applied.

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \implies A \rightarrow \alpha A', \quad A' \rightarrow \beta_1 \mid \beta_2$$

1. Decl $\rightarrow$ Type id Decl'
2. Decl' $\rightarrow ; \mid \text{Value } ;$
3. Type $\rightarrow \text{int} \mid \text{float} \mid \text{char}$
4. Value $\rightarrow \text{num} \mid \text{id}$

The factored grammar is $G_1 = (N', T, P', S)$, with $N' = \{\text{Decl}, \text{Decl}', \text{Type}, \text{Value}\}$, the same terminal set T as G_0 , the start symbol $S = \text{Decl}$, and P' made up of the productions above. N' includes the new nonterminal Decl' introduced by the factoring.

The resulting grammar no longer requires left factoring, since the common prefix of the Decl productions has been removed. It was then verified through FIRST, FOLLOW, and the predictive table that it has no conflicts for this example.

FIRST, FOLLOW, LL(1) Table, and Trace.

The following table shows the result of the FIRST and FOLLOW operations for the grammar.

Nonterminal	FIRST	FOLLOW
Decl	{int, float, char}	{ \$ }
Decl'	{ ; , = }	{ \$ }
Type	{int, float, char}	{ id }
Value	{num, id}	{ ; }

Table 9. FIRST and FOLLOW sets computed for grammar G_1 .

The LL(1) table is defined below.

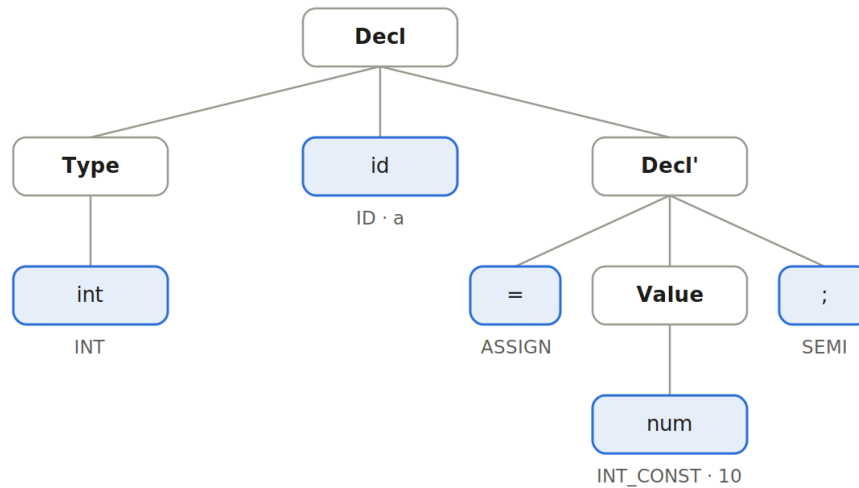
	int	float	char	id	num	=	;	\$
Decl	Decl → Type id Decl'	Decl → Type id Decl'	Decl → Type id Decl'					
Decl'						Decl' → = Value ;	Decl' → ;	
Type	Type → int	Type → float	Type → char					
Value				Value → id	Value → num			

Table 10. LL(1) predictive parsing table for grammar G_1 .

Each cell contains at most one production, so G_1 is LL(1). Trace of `int id = num ;`, with the top of the stack on the right, as in class:

Step	Stack	Input	Action
1	\$ Decl	int id = num ; \$	Decl → Type id Decl'
2	\$ Decl' id Type	int id = num ; \$	Type → int
3	\$ Decl' id int	int id = num ; \$	match int
4	\$ Decl' id	id = num ; \$	match id
5	\$ Decl'	= num ; \$	Decl' → = Value ;
6	\$; Value =	= num ; \$	match =
7	\$; Value	num ; \$	Value → num
8	\$; num	num ; \$	match num
9	\$;	; \$	match ;
10	\$	\$	accept

Table 11. Predictive parsing trace for the string `int id = num ;`



Read left to right, the leaves spell `int id = num ;` — the scan of `int a = 10;`

Figure 5: Parse tree of `int a = 10;` · grammar G_1

The leaves of the tree are the tokens `INT`, `ID`, `ASSIGN`, `INT_CONST` and `SEMI`: exactly the lexer output for test T4 (Lexical Analyzer Tests section).

Conclusions

This project delivered a lexical analyzer built from the definition of the grammars all the way to its implementation. A list of 45 tokens grouped into keywords, identifiers, constants, literals, operators, punctuation, and special characters was the first design step; regular expressions were also designed for these tokens. The regular expressions were translated into an NFA of nine sub-automata joined by ε -transitions. This automaton was then converted into a 25-state deterministic automaton using the ε -closure and *move* techniques.

The maximal munch rule, with backtracking to the last final state visited, resolves most ambiguities. Keywords and booleans did not need states of their own, because they are recognized as identifiers and classified with a lookup table. Treating + and - as independent operators avoids ambiguities in expressions such as a-1. The only real nondeterminism in the language, the / character, which can either start a comment or be an operator, was resolved in state q_8 . The unclosed block comment exception prevents the scanner from backing up and reporting a / instead of the actual error.

Lexical error handling with panic-mode recovery keeps the analyzer from stopping at the first problem. It reports the line and column and continues with the rest of the input. The staged architecture (reading, preprocessing, scanning, classification, and output), together with a symbol table and an error handler, separates responsibilities and makes it easier to modify each part independently.

Finally, the CFG for variable declarations, with its *left factoring*, FIRST and FOLLOW sets, and LL(1) table, showed how the lexical analyzer feeds the parser. The leaves of the tree for `int a = 10;` are exactly the tokens produced by the lexer, which confirms that both phases are properly connected. As future work, the lexer could be extended with scientific notation, compound operators (++ , +=), or nested comments, and the type of each identifier could be filled in the symbol table during semantic analysis.